

JEGYZŐKÖNYV

Webes adatkezelő környezetek

Féléves feladat

Autónyelvántartás

Készítette: **Jónis Bálint Rudolf**

Neptunkód: **LN40BG**

Dátum: **2025 december**

Miskolc, 2025

Bevezetés	1
A feladat leírása	2

1. Feladat	2
1.1. Az adatbázis ER modell tervezése	2
1.2. Az adatbázis konvertálása XDM modellre	3
1.3. Az XDM modell alapján XML dokumentum készítése	5
1.4. Az XML dokumentum alapján XMLSchema készítése	5
2. Feladat	8
1. Adatolvasás	8
2. Adat-lekérdezés	9
3. Adatmódosítás	12

Bevezetés

A feladat leírása

A projekt egy **autónyilvántartó rendszer** adatait kezeli. A nyilvántartás célja, hogy áttekinthető módon összekapcsolja az autókhoz tartozó tulajdonosokat, biztosításokat, baleseteket, a balesetek után a javító szervizeket, valamint a balesetekben részt vevő járműveket és azok szerepét a balesetekben.

Az XML dokumentum hierarchikus felépítésű, a gyökérelem az *Autonyilvantartas*, amely alatt külön elemekben jelennek meg a tulajdonosok, az autók, a biztosítások, a balesetek, a szervizek és a részvételek. Az adatok között logikai kapcsolatok vannak, például egy autó egy tulajdonoshoz tartozik, egy biztosítás egy konkrét autóra vonatkozik, illetve egy balesethez több autó is kapcsolódhat a részvétel elemen keresztül.

A megoldás valós életből vett példákon keresztül mutatja be egy egyszerű, de összefüggő adatmodell működését, amely alkalmas további feldolgozásra, lekérdezésre és módosításra DOM alapú programozással.

1. Feladat

1.1. Az adatbázis ER modell tervezése

A modell központi eleme az **Autó**, amelyhez több más egyed kapcsolódik különböző kapcsolattípusokon keresztül.

Az **Tulaj** egyed az autók tulajdonosait reprezentálja. Egy tulajdonoshoz több autó is tartozhat (1:N kapcsolat), ezért az Autó egyed tartalmazza a tulajdonos azonosítóját idegen kulcsként. A Tulaj rendelkezik összetett attribútummal (Cím: város, utca, házszám) és többértékű attribútummal (Telefonszám).

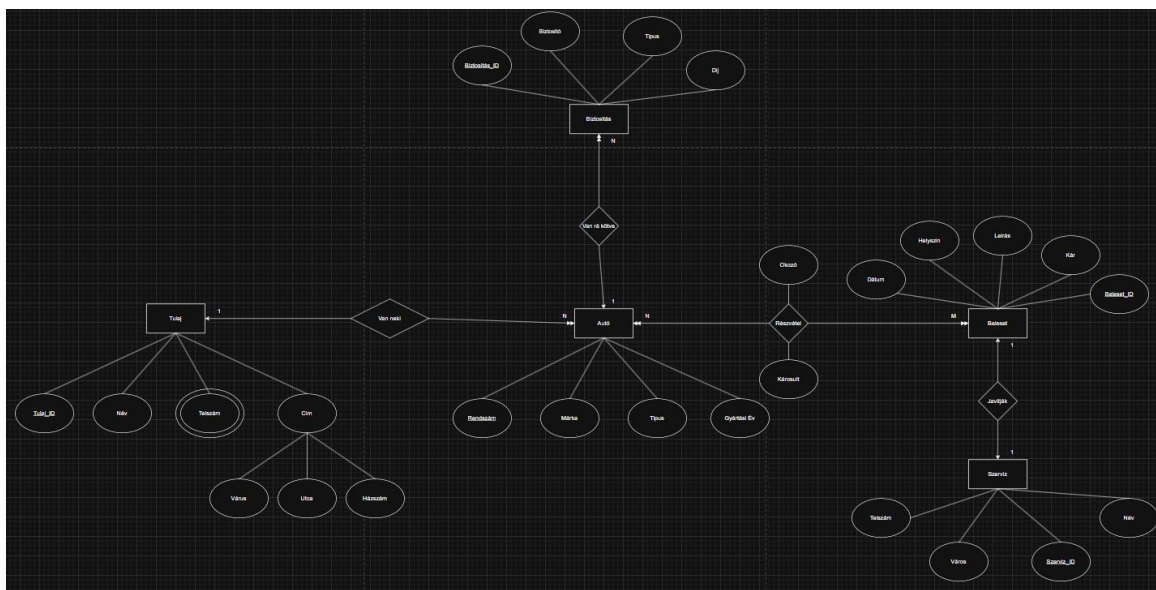
Az **Autó** egyed kulcsattribútuma a Rendszám, amely egyértelműen azonosítja a járművet. Az autóhoz kapcsolódnak a Biztosítások, ahol egy autóhoz több biztosítás is tartozhat (1:N kapcsolat).

A **Baleset** egyed az autókat érintő eseményeket írja le. Egy autó több balesetben is részt vehet, és egy baleset több autót is érinthet, ezért az Autó és a Baleset között M:N kapcsolat van, amelyet a Részvétel kapcsoló egyed valósít meg. A Részvétel tartalmazza az okozó/károsult szerepköröket.

A **Szerviz** egyed a balesetekhez kapcsolódik, minden szervizbejegyzés egy konkrét balesethez tartozik (1:1 kapcsolat a Baleset felől).

A modell tartalmaz:

- kulcsattribútumokat (Tulaj_ID, Rendszám, Bizt_ID, Baleset_ID, Szerviz_ID),
- idegen kulcsokat,
- összetett attribútumot (Cím),
- többértékű attribútumot (Telefonszám),
- valamint 1:N, M:N és 1:1 kapcsolatokat.

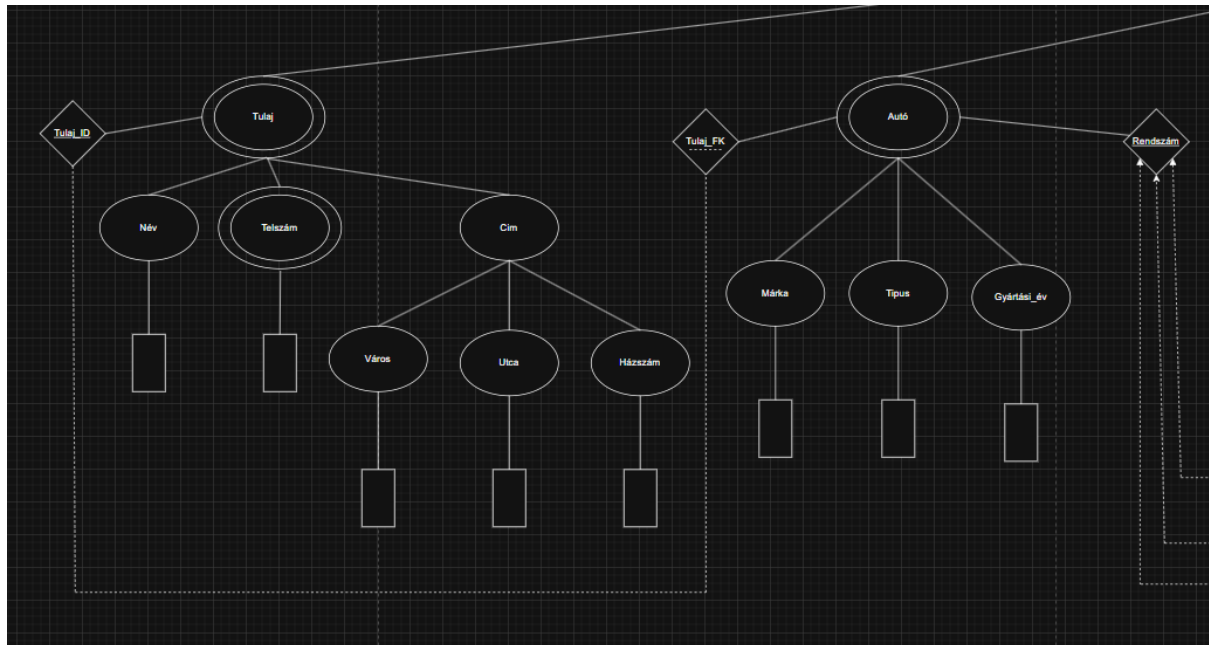


1. Ábra: ER modell

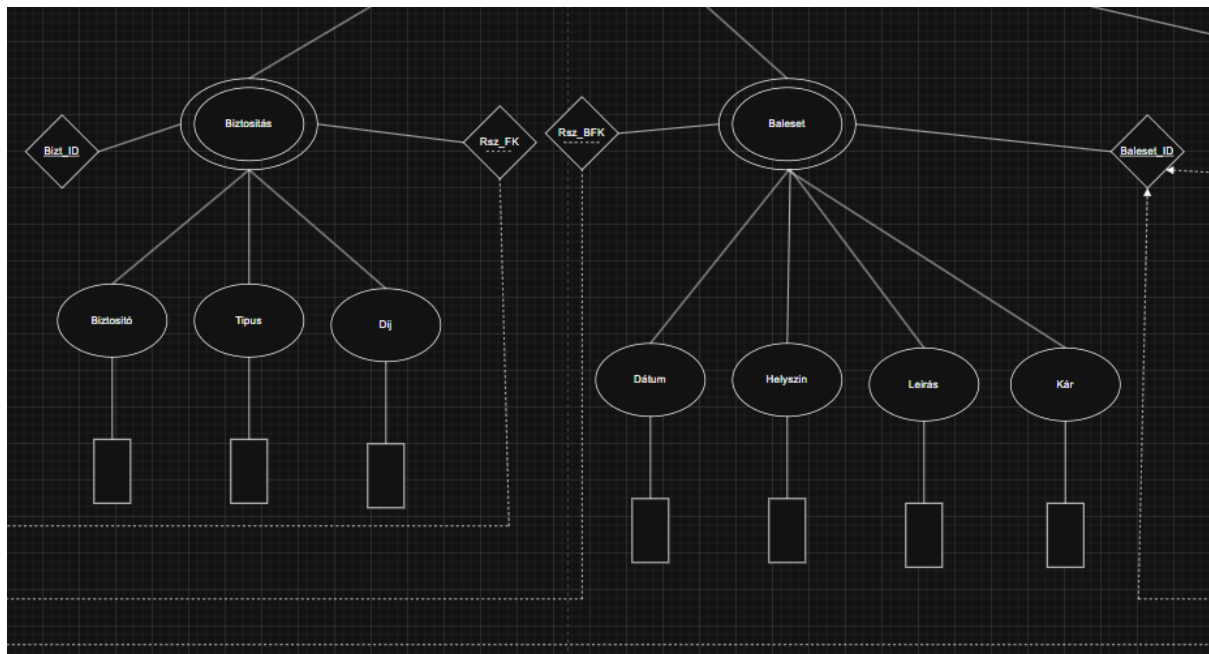
1.2. Az adatbázis konvertálása XDM modellre

A konverzió során az egyedek önálló XML elemekként jelennek meg (Tulaj, Auto, Biztositas, Baleset, Szerviz, Reszvetel), az attribútumok pedig ezek gyerekelemeként szerepelnek. Az elsődleges kulcsok külön azonosító elemek formájában kerültek megadásra (pl. Tulaj_ID, Rendszám, Baleset_ID), míg az idegen kulcsok hivatkozásokkal biztosítják az egyedek közötti kapcsolatokat.

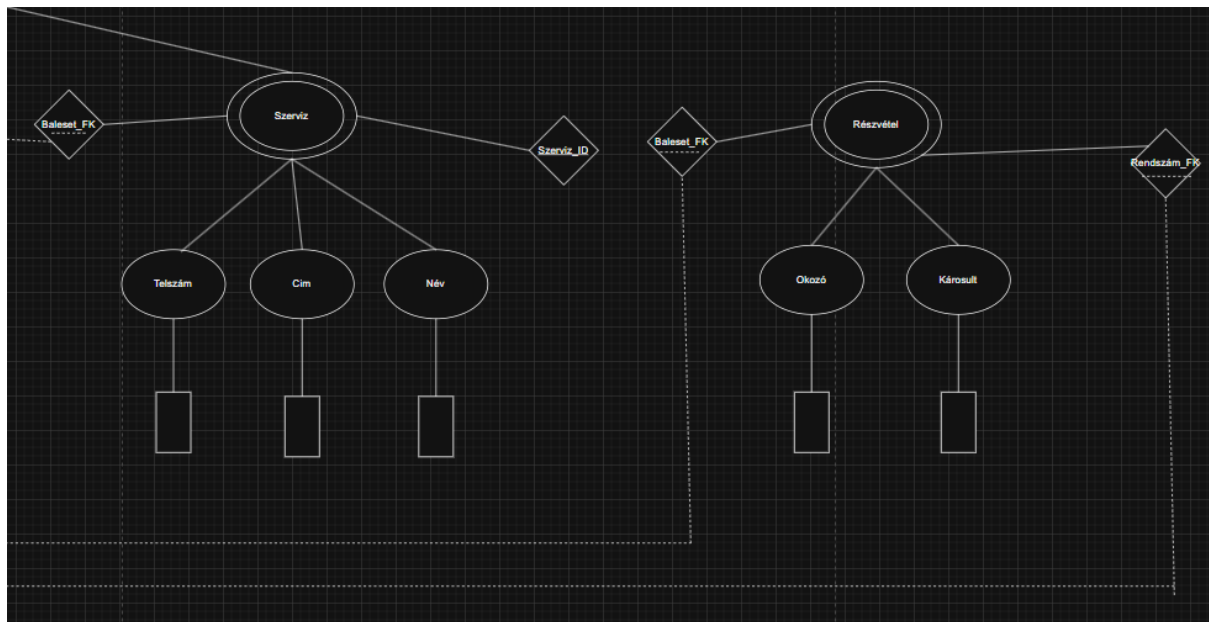
A modellben a kapcsolatok az ER modell logikáját követik: a Tulaj–Auto kapcsolat 1:N, a Baleset–Auto kapcsolat 1:N, a Reszvetel elem pedig M:N kapcsolatot valósít meg az autók és balesetek között. A Szerviz és Baleset között 1:1 kapcsolat szerepel, amelyet az idegen kulcs és az egyediség biztosít. Az összetett attribútumok (Cim) beágyazott struktúráként, a többértékű attribútumok (Telszam) többszörösen előforduló XML elemekként jelennek meg.



2. Ábra: XDM – Tulaj + Autó



3. Ábra: XDM - Biztosítás + Baleset



4. Ábra: XDM – Szerviz + Részvétel

1.3. Az XDM modell alapján XML dokumentum készítése

Az XML dokumentum a modell fő egyedeit és azok kapcsolatait hierarchikus formában írja le. A gyökérelém az Autonyilvantartás, amely alatt elkülönítve jelennek meg a tulajdonosok, autók, biztosítások, balesetek, szervizek és a baleseti részvételek.

A dokumentum tartalmazza az egyedek azonosítóit (ID-k), az idegen kulcsokkal megvalósított kapcsolódásokat, valamint az XDM modellben szereplő összetett és többértékű attribútumokat. Ilyen például a **Cím**, amely város–utca–házszám elemekre bontva szerepel, illetve a **Telszám**, amely egy tulajdonoshoz több példányban is előfordulhat.

Minden többszörösen előforduló elemhez (Tulaj, Auto, Biztositas, Baleset, Szerviz, Reszvetel) legalább két példány került rögzítésre.

Tulaj, összetett címmel:

```
<Tulaj>
  <Tulaj_ID>T1</Tulaj_ID>
  <Nev>Kiss Ádám</Nev>
  <Telszam>0620123456</Telszam>
  <Telszam>0670123456</Telszam>
  <Cim>
    <Varos>Miskolc</Varos>
    <Utca>Petőfi</Utca>
    <Hatszam>10</Hatszam>
  </Cim>
</Tulaj>
```

1.4. Az XML dokumentum alapján XMLSchema készítése

A különböző adategységekhez (Tulaj, Auto, Biztositas, Baleset, Szerviz, Reszvetel) saját komplex típusok vannak létrehozva (TulajType, AutoType, stb.), ezek határozzák meg, hogy egy elem milyen al-elemeket és milyen sorrendben tartalmazhat.

A saját egyszerű típusok az elején találhatók (pl. RendszamType, PenzType, EvType), ezekkel történik az adatok típus- és formaellenőrzése, például a rendszám formátuma vagy a pénzértékek kezelése.

```
<!-- Rendszám formátum: 3 nagybetű, kötőjel, 3 szám -->
<xs:simpleType name="RendszamType">
  <xs:restriction base="xs:string">
    <xs:pattern value="[A-Z]{3}-[0-9]{3}" />
  </xs:restriction>
</xs:simpleType>
```

Összetett típusok xs:complexType segítségével vannak megvalósítva, például:

```
<!-- Cím típus -->
<xs:complexType name="CimType">
  <xs:sequence>
    <xs:element name="Varos" type="xs:string"/>
    <xs:element name="Utca" type="xs:string"/>
    <xs:element name="Haszam" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
```

...

```
<!-- Tulaj típus -->
<xs:complexType name="TulajType">
  <xs:sequence>
    <xs:element name="Tulaj_ID" type="IDType"/>
    <xs:element name="Nev" type="xs:string"/>
    <!-- Többértékű tulajdonság: több Telszam egy Tulajnál -->
    <xs:element name="Telszam" type="TelefonType" minOccurs="1" maxOccurs="unbounded"/>
    <xs:element name="Cim" type="CimType"/>
  </xs:sequence>
</xs:complexType>
```

...

```
<!-- Részvétel (a balesetben) -->
<xs:complexType name="ReszvetelType">
  <xs:sequence>
    <xs:element name="Baleset_FK" type="IDType"/>
    <xs:element name="Rendszam_FK" type="RendszamType"/>
    <xs:element name="Okozo" type="xs:string"/>
    <xs:element name="Karosult" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
```

A ref használata az Autonyilvantartas gyökérelem alatt jelenik meg, ahol az egyes elemek (Tulaj, Auto, stb.) többször is előfordulhatnak (maxOccurs="unbounded").

Az elsődleges kulcsok (PK) az xs:key definíciókkal vannak megadva, például:

- Tulaj → Tulaj_ID
- Auto → Rendszam
- Biztositas → Bizt_ID
- Baleset → Baleset_ID
- Szerviz → Szerviz_ID

```

<!-- Tulaj primary key: Tulaj_ID -->
<xs:key name="Tulaj_PK">
  <xs:selector xpath="Tulaj"/>
  <xs:field xpath="Tulaj_ID"/>
</xs:key>

```

Az idegen kulcsok (FK) az xs:keyref elemekkel vannak definiálva, például:

- az autó tulajdonosa (Auto_Tulaj_FK)
- biztosítás és baleset kapcsolata az autóhoz
- szerviz és részvétel kapcsolata a balesethez

```

<!-- Autó.Tulaj_FK -> Tulaj.Tulaj_ID -->
<xs:keyref name="Auto_Tulaj_FK" refer="Tulaj_PK">
  <xs:selector xpath="Auto"/>
  <xs:field xpath="Tulaj_FK"/>
</xs:keyref>

```

Az 1:1 kapcsolat a Szerviz és a Baleset között az xs:unique segítségével van kikényszerítve (Szerviz_Baleset_Unique), ami biztosítja, hogy egy balesethez legfeljebb egy szerviz tartozhat.

2. Feladat

1. Adatolvasás

A program a teljes XML dokumentumot betölti, majd elemenként feldolgozza és blokkos formában írja ki a konzolra. Minden fő egyedtípus (Tulaj, Auto, Biztositas, Baleset, Szerviz, Reszvetel) külön bejárásón keresztül kerül kiírásra, így a struktúra áttekinthető marad.

A feldolgozás elején a DocumentBuilderFactory és DocumentBuilder segítségével történik az XML fájl betöltése és normalizálása.

```

// ===== XML betöltése =====
File xmlFile = new File("LN40BG_XML.xml");

DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
DocumentBuilder builder = factory.newDocumentBuilder();

Document doc = builder.parse(xmlFile);
doc.getDocumentElement().normalize();

```

Az adatok kiolvasása a getElementByTagName() metódussal történik, amely az adott XML elemeket NodeList formájában adja vissza. A program ezeket a listákat ciklussal járja be. Minden csomópontnál ellenőrzésre kerül, hogy valóban elem típusú-e (Node.ELEMENT_NODE), majd a Node objektum Element típusra kerül átalakításra, hogy az

alárendelt elemek (getElementsByTagName().item(0).getTextContent()) biztonságosan kiolvashatók legyenek. A többértékű elemek kezelése külön is megjelenik, például a Tulaj elemhez tartozó több Telszam feldolgozása NodeList segítségével történik, ciklussal kiírva az összes előfordulást.

Pl a Tulajok kiolvasása ciklussal (többi is ugyanigy):

```
for (int i = 0; i < tulajList.getLength(); i++) {
    Node node = tulajList.item(i);

    if (node.getNodeType() == Node.ELEMENT_NODE) {
        Element elem = (Element) node;

        System.out.println("Tulaj_ID: " +
elem.getElementsByTagName("Tulaj_ID").item(0).getTextContent());
        System.out.println("Név: " +
elem.getElementsByTagName("Nev").item(0).getTextContent());

        NodeList telszamok = elem.getElementsByTagName("Telszam");
        System.out.print("Telszám(ok): ");

        // Telszámok
        for (int t = 0; t < telszamok.getLength(); t++) {
            System.out.print(telszamok.item(t).getTextContent());
            if (t < telszamok.getLength() - 1) System.out.print(", ");
        }

        // Cím
        Element cim = (Element) elem.getElementsByTagName("Cim").item(0);
        System.out.println("\n" + "Cím: " +
            cim.getElementsByTagName("Varos").item(0).getTextContent() + ", " +
            cim.getElementsByTagName("Utca").item(0).getTextContent() + " " +
            cim.getElementsByTagName("Haszam").item(0).getTextContent() + "\n"
        );
    }
}
```

2. Adat-lekérdezés

Ebben az osztályban a DOM API segítségével történik az XML dokumentum célzott lekérdezése. A program elején az XML fájl betöltése és normalizálása megegyezik az adatolvasásnál használt megoldással, ez biztosítja az egységes DOM-fát, amelyen a további műveletek történnek.

Az első lekérdezés az összes autó adatát dolgozza fel. A `getElementsByTagName("Auto")` hívás visszaadja az összes `<Auto>` elemet egy `NodeList` formájában. A lista bejárása során minden csomópont típusellenőrzésen esik át, majd `Element` típusú objektumként kerül feldolgozásra. Az adott autóhoz tartozó rendszám, márka és típus a megfelelő alelemek szöveges tartalmának kiolvasásával kerül a konzolra.

```
System.out.println("1. Autók listája (rendszám + márka + típus):");
NodeList autoList = doc.getElementsByTagName("Auto");

for (int i = 0; i < autoList.getLength(); i++) {
    Node node = autoList.item(i);

    if (node.getNodeType() == Node.ELEMENT_NODE) {
        Element auto = (Element) node;

        String rendszam =
auto.getElementsByTagName("Rendszam").item(0).getTextContent();
        String marka = auto.getElementsByTagName("Marka").item(0).getTextContent();
        String tipus = auto.getElementsByTagName("Tipus").item(0).getTextContent();

        System.out.println("- " + rendszam + " | " + marka + " | " + tipus);
    }
}
```

A második lekérdezés a balesetek közül szűrést végez a kárösszeg alapján. A program minden `<Baleset>` elemet beolvas, majd a `<Kar>` elem értékét számmá alakítja. A feltételvizsgálat csak azokat a baleseteket engedi tovább, ahol a kár összege meghaladja a 200 000 forintot. Ezeknél az azonosító és a helyszín kerül kiírásra.

```
NodeList balList = doc.getElementsByTagName("Baleset");

for (int i = 0; i < balList.getLength(); i++) {
    Node node = balList.item(i);

    if (node.getNodeType() == Node.ELEMENT_NODE) {
        Element b = (Element) node;

        int kar = Integer.parseInt(b.getElementsByTagName("Kar").item(0).getTextContent());

        if (kar > 200000) {
            String id = b.getElementsByTagName("Baleset_ID").item(0).getTextContent();
            String hely = b.getElementsByTagName("Helyszin").item(0).getTextContent();
        }
    }
}
```

```

        System.out.println(" - Baleset ID: " + id + " | Helyszín: " + hely + " | Kár: " + kar);
    }
}
}

```

A harmadik lekérdezés a szervizek és a hozzájuk tartozó balesetek kapcsolatát jeleníti meg. A <Szerviz> elemek feldolgozása során a szerviz neve és a hozzá tartozó baleset idegen kulcsa (Baleset_FK) kerül kiolvasásra, így közvetlenül látható, hogy melyik szerviz melyik balesethez kapcsolódik.

```

for (int i = 0; i < szervizLista.getLength(); i++) {
    Node node = szervizLista.item(i);

    if (node.getNodeType() == Node.ELEMENT_NODE) {
        Element sz = (Element) node;

        String nev = sz.getElementsByTagName("Nev").item(0).getTextContent();
        String balesetFK = sz.getElementsByTagName("Baleset_FK").item(0).getTextContent();

        System.out.println(" - Szerviz: " + nev + " | Baleset ID: " + balesetFK);
    }
}

```

A negyedik lekérdezés egy konkrét balesethez (BA1) tartozó részvételeket szűri ki. A program minden <Reszvetel> elemet végignéz, majd a baleset azonosító alapján kiválasztja azokat, amelyek a megadott balesethez tartoznak. Ezeknél az érintett autó rendszáma, valamint az okozó és károsult státusz kerül megjelenítésre.

```

for (int i = 0; i < reszvList.getLength(); i++) {
    Node node = reszvList.item(i);

    if (node.getNodeType() == Node.ELEMENT_NODE) {
        Element r = (Element) node;

        String balFK = r.getElementsByTagName("Baleset_FK").item(0).getTextContent();

        if (balFK.equals("BA1")) {
            String rsz = r.getElementsByTagName("Rendszam_FK").item(0).getTextContent();
            String okozo = r.getElementsByTagName("Okozo").item(0).getTextContent();
            String karosult = r.getElementsByTagName("Karosult").item(0).getTextContent();

            System.out.println(" - Autó: " + rsz + " | Okozó: " + okozo + " | Károsult: " + karosult);
        }
    }
}

```

```
}  
}
```

A lekérdezések közös jellemzője, hogy minden esetben NodeList-en keresztül történik a feldolgozás, a tényleges adatkinyerés pedig az elemek hierarchiájának DOM-alapú bejárásával valósul meg. Ez biztosítja, hogy a lekérdezések kizárólag az XML dokumentum aktuális tartalmára épüljenek.

3. Adatmódosítás

Az adatmódosítás DOM alapon történik, az XML dokumentum teljes betöltése után. A program a DocumentBuilder segítségével felépíti a DOM fát, majd célzottan megkeresi azokat az elemeket, amelyeken módosítást kell végrehajtani. A módosítások közvetlenül a DOM objektumon történnek, `setTextContent()` hívásokkal.

A módosítás minden esetben az adott elem NodeList-jének lekérésével indul (`getElementsByTagName()`), majd az első találat Element típusra kerül, amelyen a megfelelő gyermekelem értéke frissítésre kerül.

Az első módosítás egy Tulaj elemhez tartozó név megváltoztatása. A program a Tulaj elemek listájából kiválasztja az első tulajdonost, majd a Nev elem szöveges tartalmát módosítja.

```
NodeList tulajList = doc.getElementsByTagName("Tulaj");  
Element tulaj = (Element) tulajList.item(0);  
tulaj.getElementsByTagName("Nev").item(0).setTextContent("Kiss Ádám Jr.");
```

A második módosítás egy Auto elemhez tartozó gyártási év átírása. Az Auto elem első példányán a `Gyartasi_ev` értéke kerül frissítésre, ami jól mutatja, hogyan kezelhetők numerikus adatok is szövegként a DOM modellben.

```
NodeList autoList = doc.getElementsByTagName("Auto");  
Element auto = (Element) autoList.item(0);  
auto.getElementsByTagName("Gyartasi_ev").item(0).setTextContent("2019");
```

A harmadik módosítás egy Biztositas elemhez kapcsolódik. Itt a biztosítás díja (Dij) kerül módosításra, szintén közvetlenül az adott elem szöveges tartalmának felülírásával.

```
NodeList biztList = doc.getElementsByTagName("Biztositas");  
Element bizt = (Element) biztList.item(0);  
bizt.getElementsByTagName("Dij").item(0).setTextContent("27000");
```

A negyedik módosítás egy Baleset elem kárösszegét érinti. A Kar mező új értéket kap, amely demonstrálja, hogy üzleti adat módosítása is ugyanazzal a DOM művelettel történik, mint bármely más mezőé.

```
NodeList balList = doc.getElementsByTagName("Baleset");  
Element baleset = (Element) balList.item(0);  
baleset.getElementsByTagName("Kar").item(0).setTextContent("350000");
```

Minden módosítás után az érintett elem azonosítója és az új érték kiírásra kerül a konzolra, blokkos formában, így egyértelműen követhető, hogy mely adatok változtak meg.